



Northeastern University
College of Professional Studies

Module 4

Using an Off-the-shelf Model

EAI6010: Applications of Artificial Intelligence

Cheng Liu

Professor : Wen Sun

14th March 2025

Table of Contents

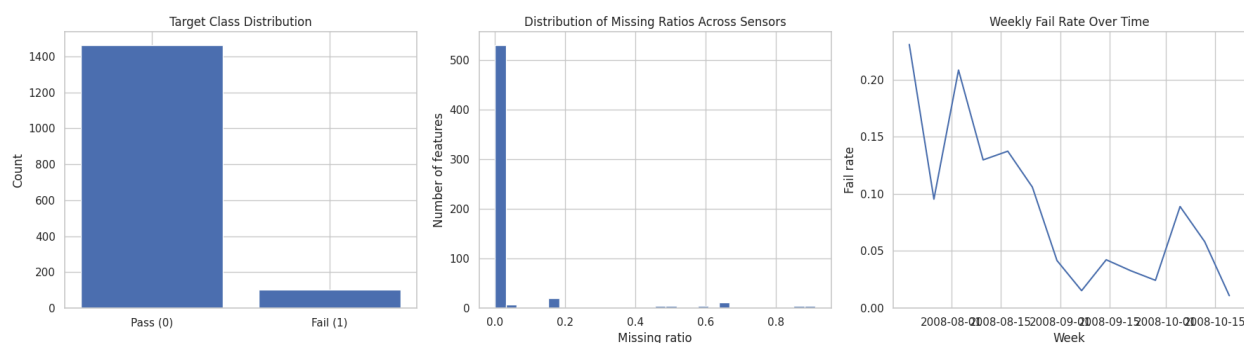
How did I pick the dataset?.....	3
Figure 1. Target class distribution + missing-ratio distribution + weekly fail-rate trend.....	3
What changes did I need to make to use it?.....	3
What difficulties did I encounter, and how did I address them?.....	4
Once functional, did the implementation result in a model I would feel comfortable deploying?.....	5
Figure 2. MLP training loss and validation balanced accuracy.....	6
Figure 3. Test confusion matrix – Random Forest.....	7
Figure 4. ROC curve – Random Forest.....	8
Figure 5. Precision-recall curve – Random Forest.....	9
Figure 6. Top 20 Random Forest feature importances.....	10
Conclusion.....	11
Reference.....	11

How did I pick the dataset?

I picked the UCI SECOM dataset because I wanted a semiconductor-related problem that was closer to a real manufacturing quality-monitoring scenario. I was originally inspired by a Kaggle tutorial about semiconductor and IoT modeling, but that tutorial was not a direct fit for this assignment. I needed a dataset that I could adapt to my own learning problem.

The SECOM dataset includes sensor measurements and a pass/fail label, so it fits a realistic defect-screening or quality-risk prediction setting. From an AI and analytics perspective, it is also a good example of a hard industrial dataset: it has high dimensionality, missing values, and a strongly imbalanced target. That made it a useful dataset for testing whether an off-the-shelf modeling idea could be adapted into a more careful and explainable workflow.

Figure 1. Target class distribution + missing-ratio distribution + weekly fail-rate trend



What changes did I need to make to use it?

The main change was that I did not directly copy an image-based workflow. The SECOM dataset is tabular sensor data, so I redesigned the project around models that fit structured numeric features better.

I used a leakage-safe train/validation/test workflow. I split the data first, then fit preprocessing only on the training data. For preprocessing, I dropped columns with more than 50% missing values based on the training split only, imputed the remaining missing values with the training-set median, removed constant features, and then used scaling and PCA for the linear-model and neural-network versions.

I compared multiple models instead of depending on only one. I used:

- a Dummy Classifier as a sanity-check baseline,
- Logistic Regression with PCA as a simple interpretable baseline,
- Random Forest as a stronger nonlinear baseline,
- and a class-weighted MLP in PyTorch to stay aligned with the AI course.

Another important change was how I handled class imbalance. In the final version, I did not use random undersampling or SMOTE as the main path. Instead, I used class-weighted learning and threshold tuning. That made the workflow easier to explain and more defensible for a small and imbalanced dataset.

What difficulties did I encounter, and how did I address them?

The biggest difficulty was class imbalance. The fail class is only a small part of the dataset, so plain accuracy would look misleadingly high even if the model almost never detected failures. To deal with that, I focused more on balanced accuracy, recall, PR-AUC, ROC-AUC, and confusion-matrix interpretation instead of accuracy alone.

Another difficulty was missing data. Many sensor columns had missing values, and some had extremely high missing ratios. I addressed this by doing a missing-value audit first, then dropping features above the missingness threshold using only the training split. After that, I used median imputation for the remaining missing values.

The dataset was also high-dimensional and noisy. There were many low-information features, including constant or near-constant columns. I handled this by removing constant features and applying PCA for the Logistic Regression and MLP pipelines. This reduced dimensionality and made training more stable.

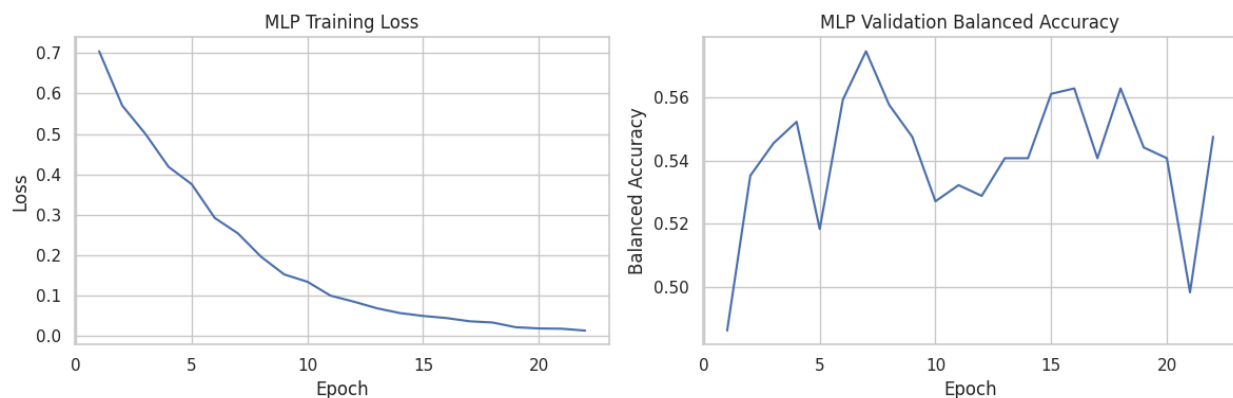
One more issue was that the timestamps suggested the process might change over time. I used the timestamps only for exploratory analysis in this version, not as model features. I still used a stratified random split for a stable baseline comparison, but I also noted that a more deployment-oriented version should test time-based validation.

Once functional, did the implementation result in a model I would feel comfortable deploying?

Not yet, but it became a much more realistic analytical prototype.

On the validation split, Random Forest gave the best balanced accuracy among the compared models. Logistic Regression + PCA was still competitive on some metrics such as F1 and PR-AUC, while the weighted MLP pushed recall very high but at the cost of very low precision and much lower overall accuracy. That result actually helped me show something important: a neural network is not automatically the best choice for this kind of small, imbalanced tabular dataset.

Figure 2. MLP training loss and validation balanced accuracy



Because Random Forest had the strongest validation balanced accuracy, I selected it for final test evaluation. On the untouched test set, it achieved:

- accuracy = 0.7898
- balanced accuracy = 0.6663
- recall = 0.5238
- ROC-AUC = 0.7978
- PR-AUC = 0.2192

From the confusion matrix, the model correctly detected 11 of the 21 fail cases, but it also produced many false positives. That means it may have some value as a screening or early-warning tool, but I would not treat it as a fully automated pass/fail decision system.

Figure 3. Test confusion matrix – Random Forest

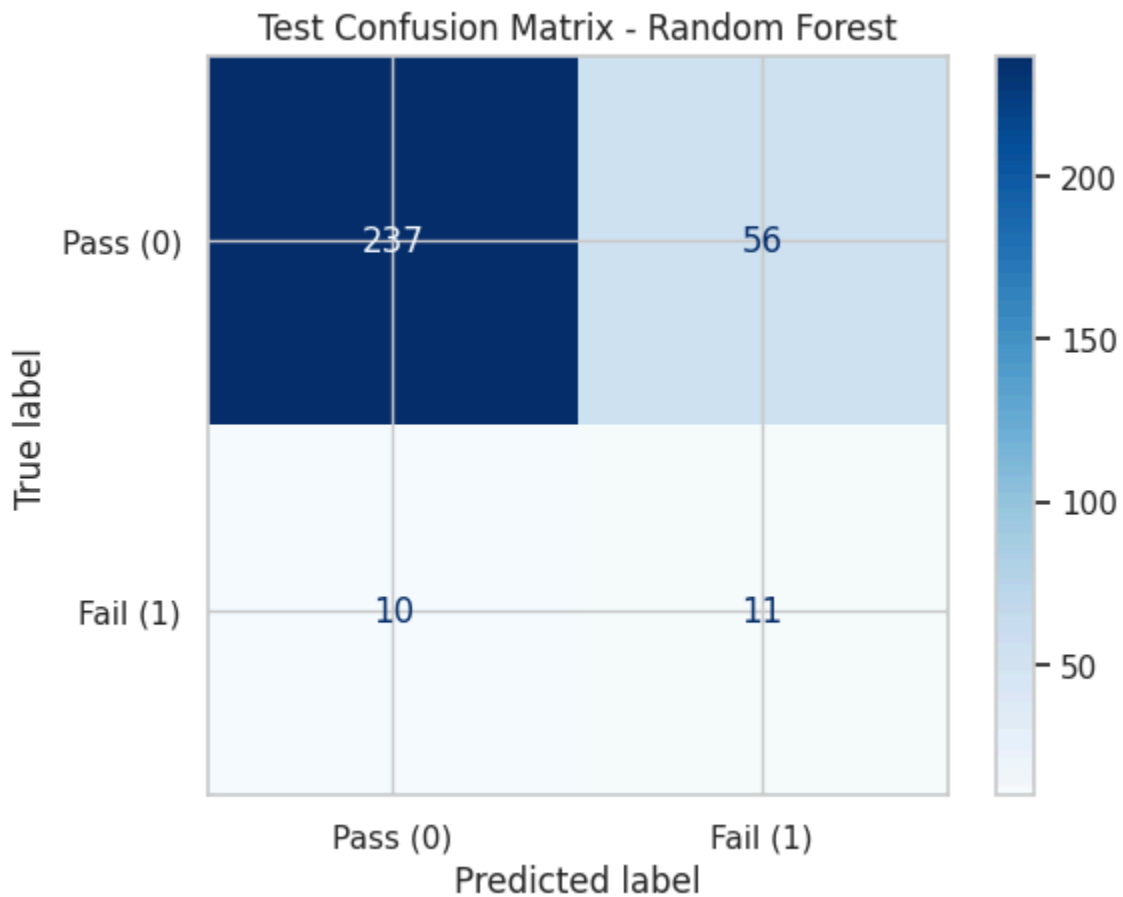


Figure 4. ROC curve – Random Forest

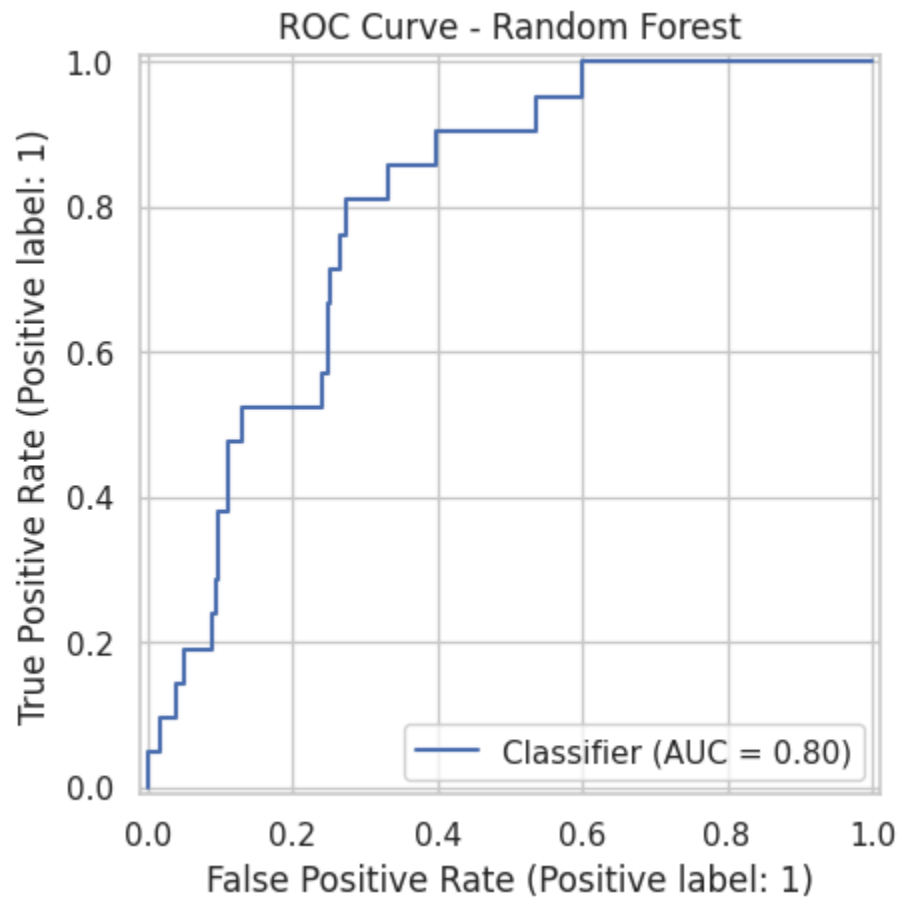
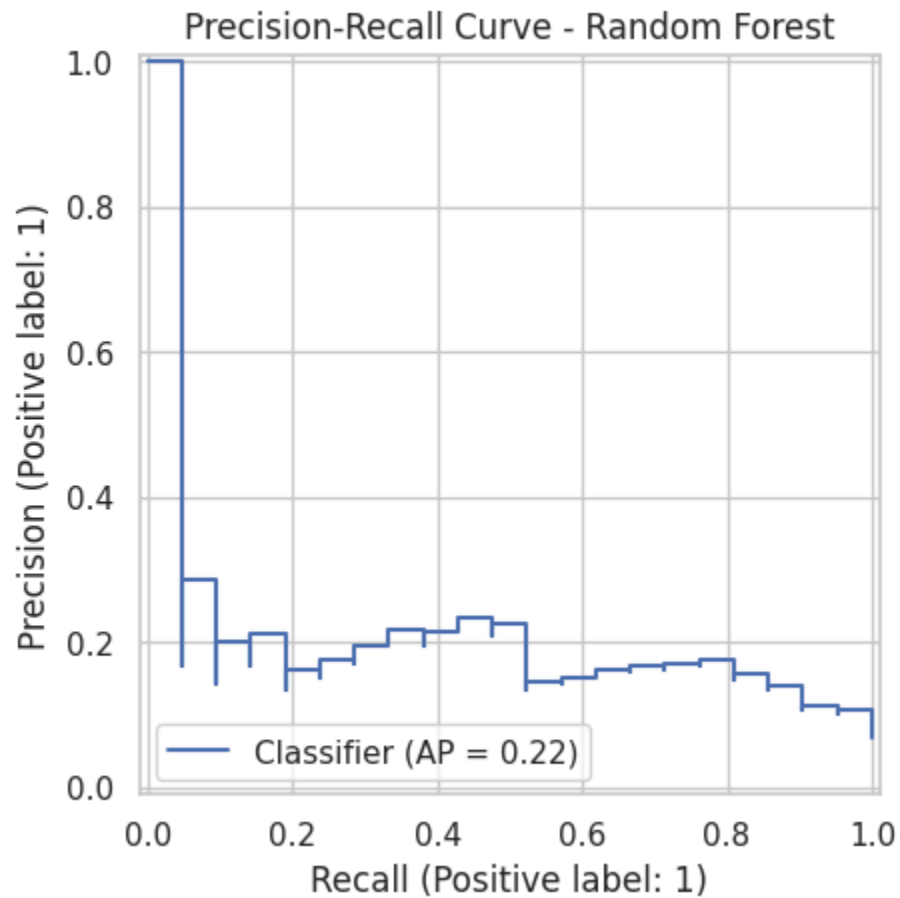
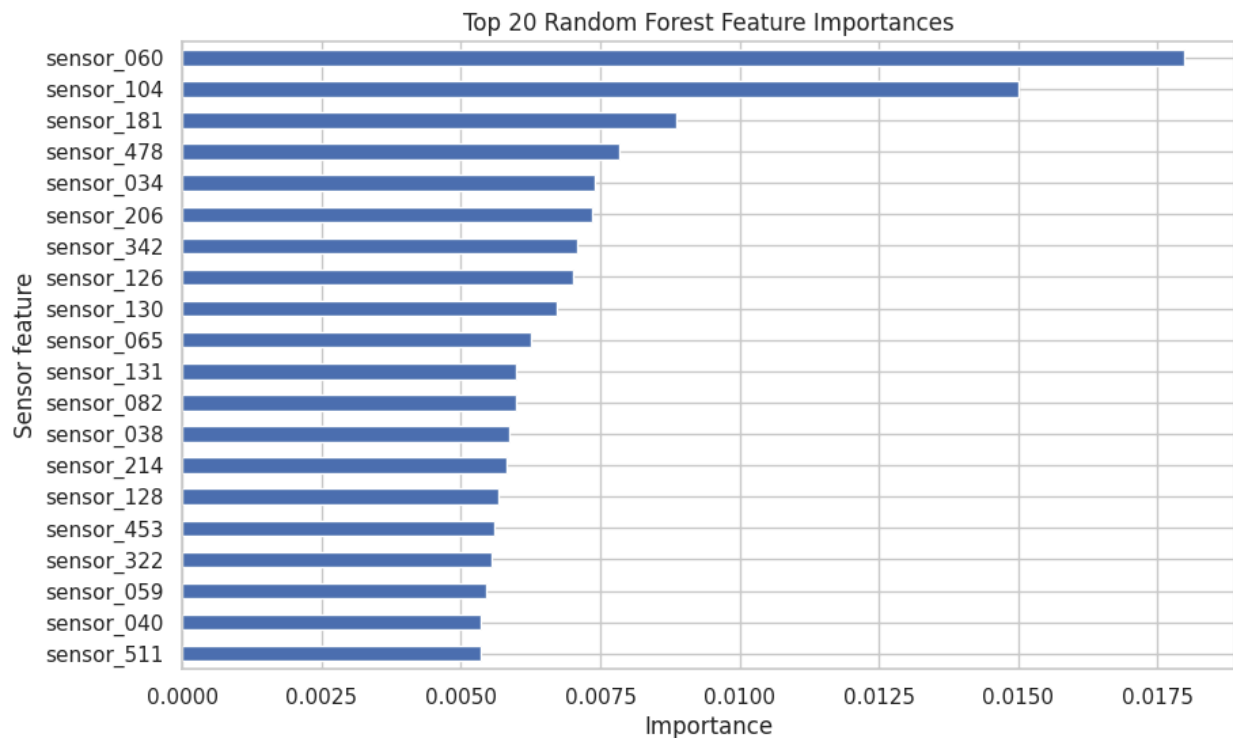


Figure 5. Precision-recall curve – Random Forest



I also reviewed the Random Forest feature importance results. These are useful for identifying which sensor variables influenced the model most on this split, but I would not claim they prove true physical root causes. In a real semiconductor setting, those signals would still need process knowledge and engineering validation.

Figure 6. Top 20 Random Forest feature importances



So overall, I would say the model is good enough as a course project and a stronger prototype, but not ready for real deployment yet. Before deployment, I would still want:

- repeated validation or cross-validation,
- time-based evaluation because timestamps are available,
- threshold selection based on business or engineering cost,
- drift monitoring,
- and a clearer understanding of whether the most important sensors are stable and operationally available at scoring time.

Conclusion

This project showed me that using an off-the-shelf model is not just about getting a model to run. The more important part is adapting the workflow to the actual data and problem setting.

For the SECOM dataset, the main challenges were class imbalance, missing values, many noisy sensor features, and possible process shift over time. I handled those issues by building a leakage-safe pipeline, comparing multiple baseline and nonlinear models, using balanced metrics, and tuning the classification threshold on the validation set.

The final result was not a production-ready semiconductor model, but it was a much more realistic and defensible AI workflow. It also matched the main idea of this module well: taking an existing modeling idea, adapting it to a new problem, and thinking beyond raw accuracy toward evaluation, limitations, and deployment concerns.

Reference

- UCI Machine Learning Repository. [SECOM Dataset](#).
- Vinit. (2024). [Semiconductor/IOT-MachineLearning](#). Kaggle.
- Howard, J., & Gugger, S. (2020). [Deep Learning for Coders with Fastai and PyTorch. Chapter 6: Other Computer Vision Problems](#). O'Reilly.